Project Progress Report: Improving RUNG for Robust Graph Aggregation

Tawkir Aziz Rahman
2005090
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh

Dipanta Kumar Roy Nobo
2005074
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh

ABSTRACT

This report presents our project progress on improving RUNG (Robust UNbiased Neighborhood Graph aggregation) for robust Graph Neural Networks under Adaptive Attack settings. We start from the RUNG 2024 NeurIPS foundation and test three improvements: Percentile Gamma, Parametric Gamma, and Learnable Cosine Distance, followed by a combined model. We compare behavior on homophilic and heterophilic datasets, discuss what is working now, and list the remaining gaps we plan to address before the final project presentation.

CCS CONCEPTS

• **Computing methodologies** → **Learning from graph data**; • **Security and privacy** → *Adversarial learning*.

KEYWORDS

Graph Neural Networks, RUNG, RUGE, Adaptive Attack, Estimation Bias, Percentile Gamma, Parametric Gamma, Learnable Distance, Heterophily

ACM Reference Format:
Tawkir Aziz Rahman, 2005090 and Dipanta Kumar Roy Nobo, 2005074. 2026. Project Progress Report: Improving RUNG for Robust Graph Aggregation. In . ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

The base paper for this project is RUNG 2024 NeurIPS, "Robust Graph Neural Networks via Unbiased Aggregation" [2]. RUNG means Robust UNbiased Neighborhood Graph aggregation. It is designed for robust graph aggregation techniques for Graph Neural Networks.

The core problem is that many GNNs can face adaptive attacks. Attackers can add fake edges among nodes and disrupt the result. Many graph signal estimators exist and do work, but they fail badly in one case: when the attacker knows the model and the dataset. If the attacker knows the parameters, they can attack in an adaptive way and break those defenses.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Previous defenses include two regularization styles, often described as ℓ_1 and ℓ_2 defense. The ℓ_1 defense uses an absolute penalty, while ℓ_2 uses a squared penalty. In both styles, the algorithm checks edges and assigns weights. Suspicious edges get larger penalties so they cannot hurt the graph as much.

The problem appears when the attack budget is high, meaning the attacker can inject many fake edges. Even after weighting and penalizing, those fake edges still influence the final aggregation and shift the estimate. This is the Estimation Bias problem [1]. For ℓ_1 penalty, the values still pull the estimate over time. For ℓ_2, the shift is often worse because squared penalties can deviate further from the original signal.

RUNG addresses this with an unbiased strategy. Instead of only reducing suspicious edge weights, it can force them to zero if they are highly suspicious. The goal is Unbiased Aggregation: if an edge crosses a suspicious threshold, sever it completely.

RUNG does this through the Robust and Unbiased Graph signal Estimator (RUGE) [2]. The estimator uses MCP-based weighting to decide whether an edge should contribute or be pruned:

$$\arg\min_{\mathbf{F}} \mathcal{H}(\mathbf{F}) = \sum_{(i,j)\in\mathcal{E}} \rho_\gamma\left(\left\|\frac{\mathbf{f}_i}{\sqrt{d_i}} - \frac{\mathbf{f}_j}{\sqrt{d_j}}\right\|_2\right) + \lambda \sum_{i\in\mathcal{V}} \|\mathbf{f}_i - \mathbf{f}_i^{(0)}\|_2^2 \tag{1}$$

$$\rho_\gamma(y) = \begin{cases} y - \dfrac{y^2}{2\gamma}, & y < \gamma \\ \dfrac{\gamma}{2}, & y \geq \gamma \end{cases} \tag{2}$$

The MCP form in Eq. 2 follows the same non-convex penalty family discussed in statistical learning [1] and is the key reason RUGE can reduce bias while preserving robust behavior. This is the foundation we built on in our project.

2 MATERIALS AND METHODS

2.1 Datasets

We evaluate on homophilic datasets (Cora, Citeseer, Pubmed) and heterophilic datasets (Cornell, Wisconsin, Chameleon). The heterophilic benchmarks are standard in Geom-GCN [5]. We included them because the base RUNG paper is mainly stronger for homophilic graphs, and we wanted to test what happens when class-connected neighbors are less similar.

2.2 Base Architecture and Training Context

Our starting point is the RUNG architecture with the RUGE estimator and MCP penalty [2]. The training setup follows the same robust aggregation idea under Adaptive Attack conditions. In our

progress stage, we focused on modifying the thresholding and distance components while keeping the core backbone stable.

2.3 Method 1: Percentile Gamma

Our first target was changing gamma in the RUGE estimator. In original RUNG, gamma is selected early and kept fixed across layers. The issue is that with deeper GNN layers, node features become smoother and pairwise differences become smaller, a standard oversmoothing behavior [4]. If gamma stays fixed, deeper layers can treat too many edges as valid.

To fix this, we implemented Percentile Gamma. Gamma is adapted per layer using a percentile-based threshold over edge differences. As differences shrink, gamma also shrinks. Attack edges usually connect dissimilar nodes, so they appear in the high-difference tail. We prune edges above a selected q-percentile rule. We can choose aggressive pruning (e.g., 50%), moderate (75%), or conservative (95%). In this report, we use $q = 0.75$ as the normal setting.

2.4 Method 2: Parametric Gamma

Second, we implemented Parametric Gamma. Percentile Gamma estimates threshold from current layer statistics. Parametric Gamma instead enforces a predictable decay curve with depth. We set a starting threshold γ_0 and a decay multiplier r, then apply:

$$\gamma_k = \gamma_0 \, r^k.$$

Example with $\gamma_0 = 5.0$ and $r = 0.8$: layer 0 is 5.0, layer 1 is 4.0, layer 2 is 3.2, layer 3 is 2.56. This smooth decay follows the same trend as feature smoothing, so the defense remains active at deeper layers where fixed thresholds fail [4].

2.5 Method 3: Learnable Cosine Distance

Third, we implemented a learnable defense in RUNG by changing the edge suspiciousness metric. Core RUNG relies on Euclidean distance, which depends strongly on magnitude. Under Adaptive Attack, a malicious node can be crafted to mimic magnitude and reduce Euclidean gap, so poisoned edges can pass through.

We moved to cosine-based distance and made it learnable through a trainable projection matrix before cosine computation. In simple terms, the model learns a custom feature space and then compares directional similarity there. This helps separate useful structural signals from noisy features. The idea is aligned with graph structure learning directions in robust GNN literature [3].

2.6 Combined Model Design

After testing each method separately, we combined them to test unified improvement. The combination is not a direct conflict between Percentile Gamma and Parametric Gamma. We do not apply parametric decay directly to raw distance threshold. Instead, we apply parametric decay to the percentile level q.

In pure Percentile Gamma, q is fixed for all layers. In the combined design, q decays with depth, so pruning gets stricter in deeper layers. Parametric Gamma controls aggressiveness over depth, while Percentile Gamma controls how many high-difference edges are pruned per layer.

Learnable Cosine Distance complements this by measuring angle-based feature inconsistency rather than magnitude-only mismatch. That made the model more robust under higher-budget attacks in several settings.

Two design choices explain why we used this exact combination:

- We did not use only Parametric Gamma + Learnable Distance because learnable distance changes scale during training, while a rigid parametric threshold can become mismatched.
- We did not use only Percentile Gamma + Learnable Distance with flat q, because this ignores depth-wise oversmoothing and can under-prune or over-prune depending on layer depth.

3 RESULTS AND DISCUSSION

3.1 Percentile Gamma Results

For Cora and Citeseer with $q = 0.75$, Percentile Gamma shows improved robustness over base RUNG when attack budget increases.

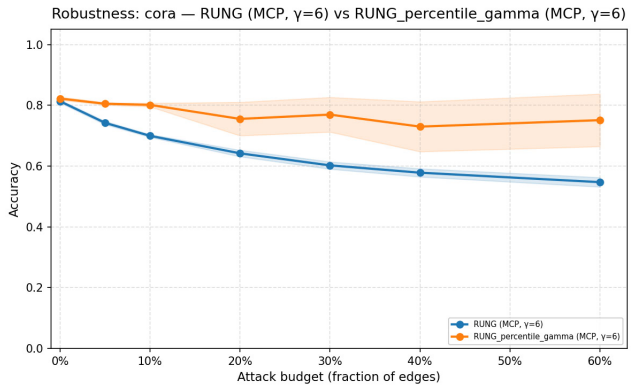

Figure 1: Cora: RUNG vs Percentile Gamma.

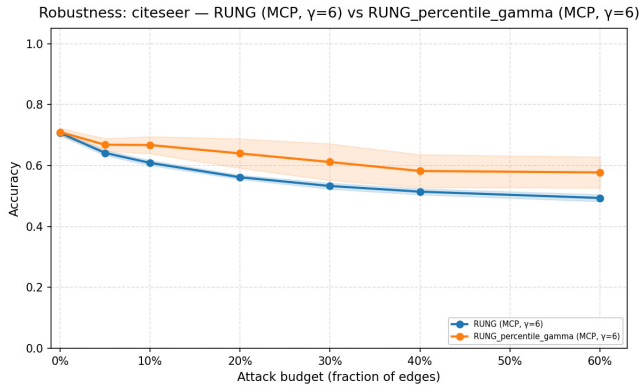

Figure 2: Citeseer: RUNG vs Percentile Gamma.

3.2 Parametric Gamma Results

On Cora and Citeseer, Parametric Gamma also improves performance over RUNG as attack budget grows. We still observe high variance in some settings, so this method needs more tuning.

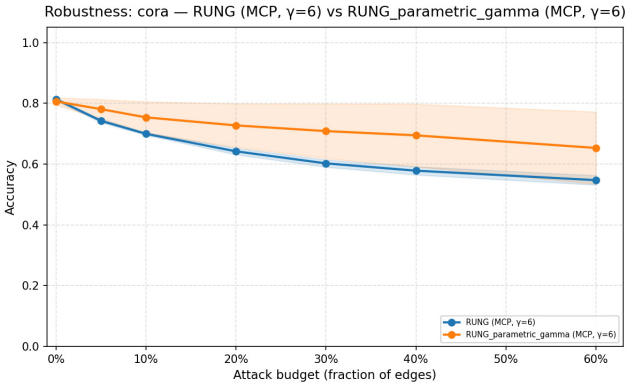

Figure 3: Cora: RUNG vs Parametric Gamma.

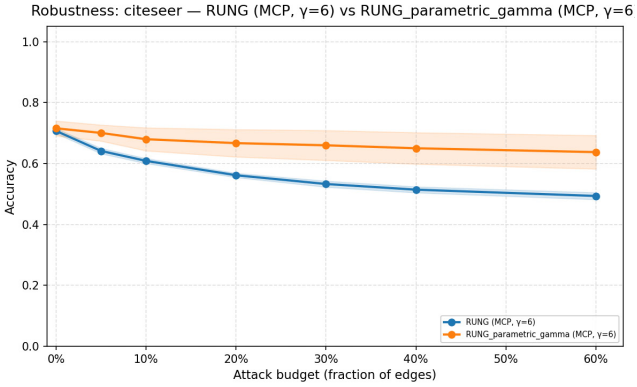

Figure 4: Citeseer: RUNG vs Parametric Gamma.

3.3 Learnable Cosine Distance Results

Learnable Cosine Distance gives good improvements in several runs. It performs especially well on heterophilic settings where angle-based similarity can preserve useful cross-class structure better than magnitude-based Euclidean distance.

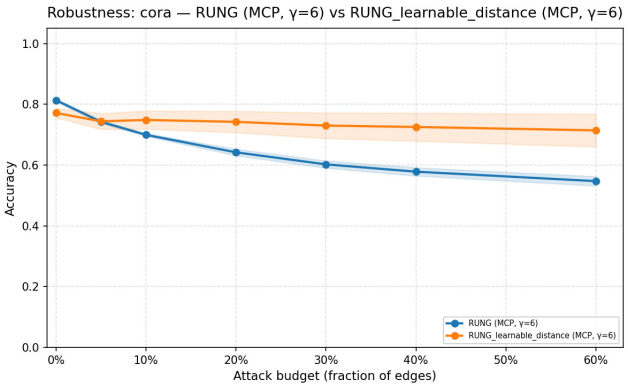

Figure 5: Cora: RUNG vs Learnable Cosine Distance.

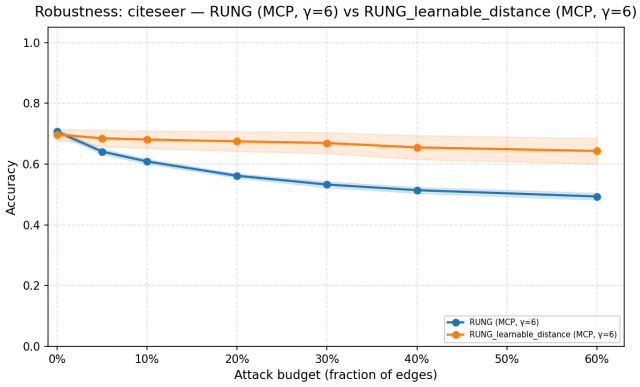

Figure 6: Citeseer: RUNG vs Learnable Cosine Distance.

3.4 Combined Model Results

The combined model was built to test whether the methods strengthen each other under stronger attacks. For homophilic datasets, the combined setup performs very well at higher attack budgets.

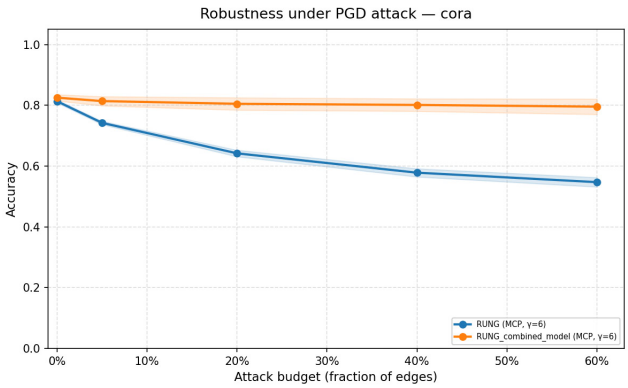

Figure 7: Cora: RUNG vs Combined Model.

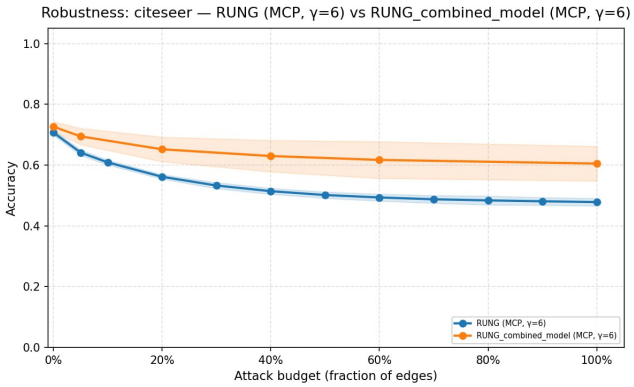

Figure 8: Citeseer: RUNG vs Combined Model.

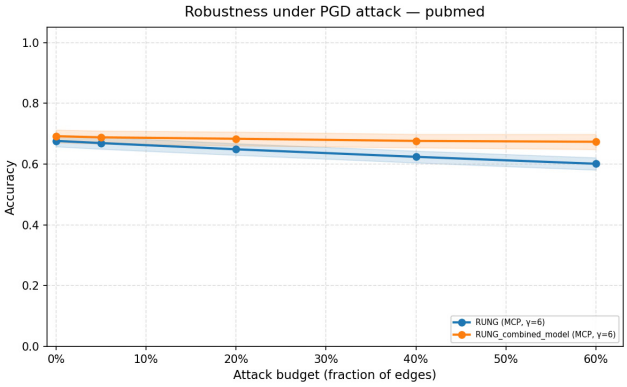

Figure 9: Pubmed: RUNG vs Combined Model.

For heterophilic datasets, we observed a different behavior. Learnable Distance can outperform combined thresholding variants in some cases. The combined model is often still better than original RUNG, but not always better than the best independent method. A key reason is over-pruning in heterophilic graphs: valid edges can naturally connect dissimilar nodes.

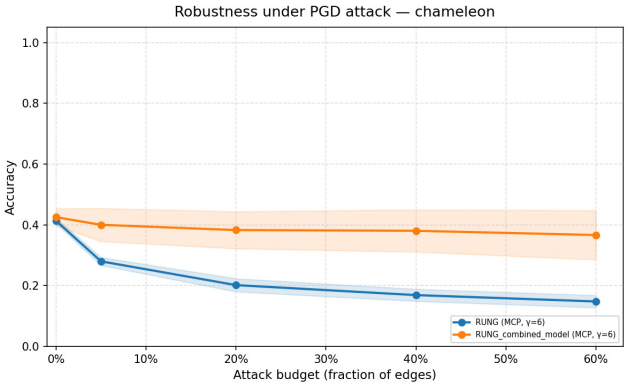

Figure 10: Chameleon: RUNG vs Combined Model.

3.5 Heterophilic Graph Analysis

The paper itself already indicates that RUNG is mainly designed for homophilic graphs. That is why we included Chameleon, Wisconsin, and Cornell [5]. In our tests, baseline RUNG performs weakly in these settings. Learnable Cosine Distance often performs better, likely because Euclidean distance fails when valid neighbors are far in magnitude but still structurally meaningful.

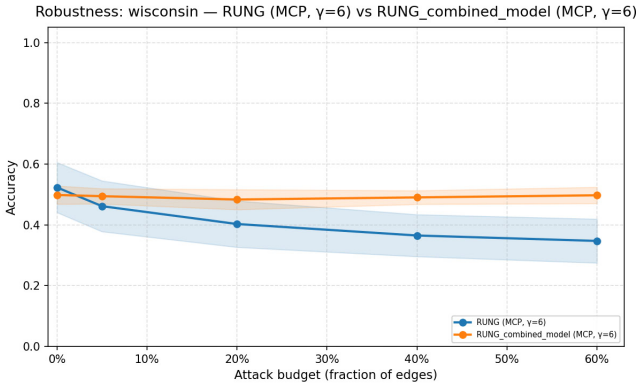

Figure 11: Wisconsin: RUNG vs Combined Model.

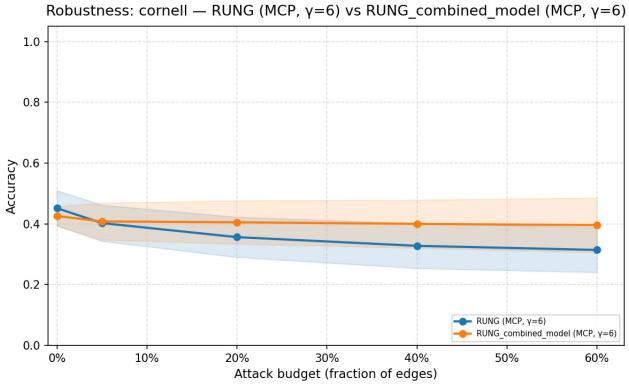

Figure 12: Cornell: RUNG vs Combined Model.

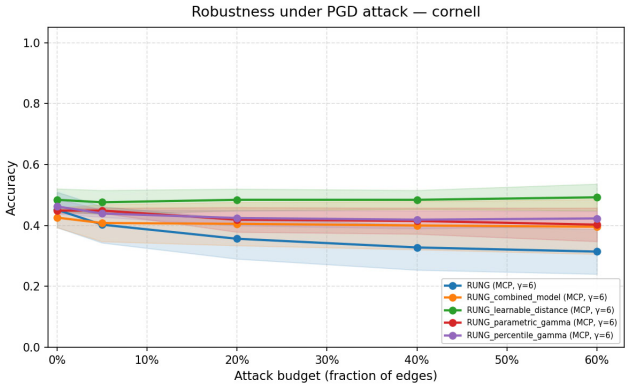

Figure 13: Cornell: All methods comparison.

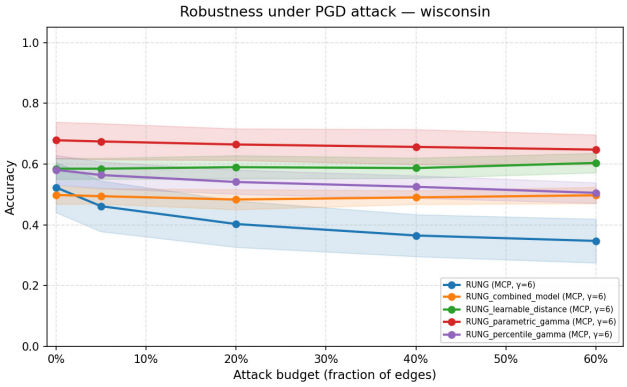

Figure 14: Wisconsin: All methods comparison.

The core issue we observed is a double-correction effect in heterophilic graphs. Learnable Distance tries to balance directional structure, but depth-wise parametric strictness can increase pruning too quickly. This can choke message passing and remove valid heterophilic edges. This explains why independent methods can beat the combined model in heterophilic space.

From this progress stage, our scientific claim is the following:

Our experiments reveal that while a combined approach is highly effective for homophilic graphs, heterophilic graphs suffer from over-regularization when dynamic thresholding (Percentile/Parametric) is combined with dynamic metrics (Learnable Distance). For heterophilic environments, upgrading the distance metric to Cosine Learnable Distance independently yields the most optimal robustness, as it prevents the accidental over-pruning of valid heterophilic edges.

4 CONCLUSION

This progress report shows that our three modifications improve robustness over base RUNG in many attack settings. Percentile Gamma and Parametric Gamma address depth-wise threshold mismatch caused by oversmoothing. Learnable Cosine Distance improves robustness by changing edge suspiciousness measurement from magnitude-driven Euclidean comparison to direction-aware similarity.

The combined model performs strongly on homophilic datasets, especially under high attack budget. For heterophilic datasets, we observed that independent Learnable Cosine Distance can outperform the combined setup because strict dynamic pruning can remove valid edges.

For future work toward the final project presentation, we plan to:

- complete broader training and testing for each method independently and in combination,
- increase attack budget analysis beyond current levels up to 100%,
- build stronger variance analysis plots for all datasets,
- and develop clearer theoretical guarantees for why each method behaves this way.

The theoretical part is still hard for us because the base paper is complex and these topics are new to us, but this is the key target before final presentation.

REFERENCES

[1] Jianqing Fan and Runze Li. 2001. Variable Selection via Nonconcave Penalized Likelihood and Its Oracle Properties. *J. Amer. Statist. Assoc.* 96, 456 (2001), 1348–1360.

[2] Zhetao Hou, Rui Feng, Tyler Derr, and Xiao Liu. 2024. Robust Graph Neural Networks via Unbiased Aggregation. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc.

[3] Wei Jin et al. 2020. Graph Structure Learning for Robust Graph Neural Networks. In *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 66–76.

[4] Qimai Li, Zhichao Han, and Xiao-Ming Wu. 2018. Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, 3538–3545.

[5] Hongbin Pei, Bo Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. 2020. Geom-GCN: Geometric Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*. OpenReview.net.